SQL Implementation of a Sample Spotify Database

Group Member Names: William Bruseo, Jianna Fernandez, Jeremy Batista, Alvin Lai

- High Level Description:

    This data displays one of the large scale music data sets derived from Spotify track data. This system was created to keep information about artists, albums, songs, and detailed audio characteristics of every track.

    The Artist entity stores the artist's identifier and names. The Album entity stores album information and is linked to the artists, which allows an artist to have multiple albums. The Song entity represents each track and connects to both the artists who created the song and which album it appears in.

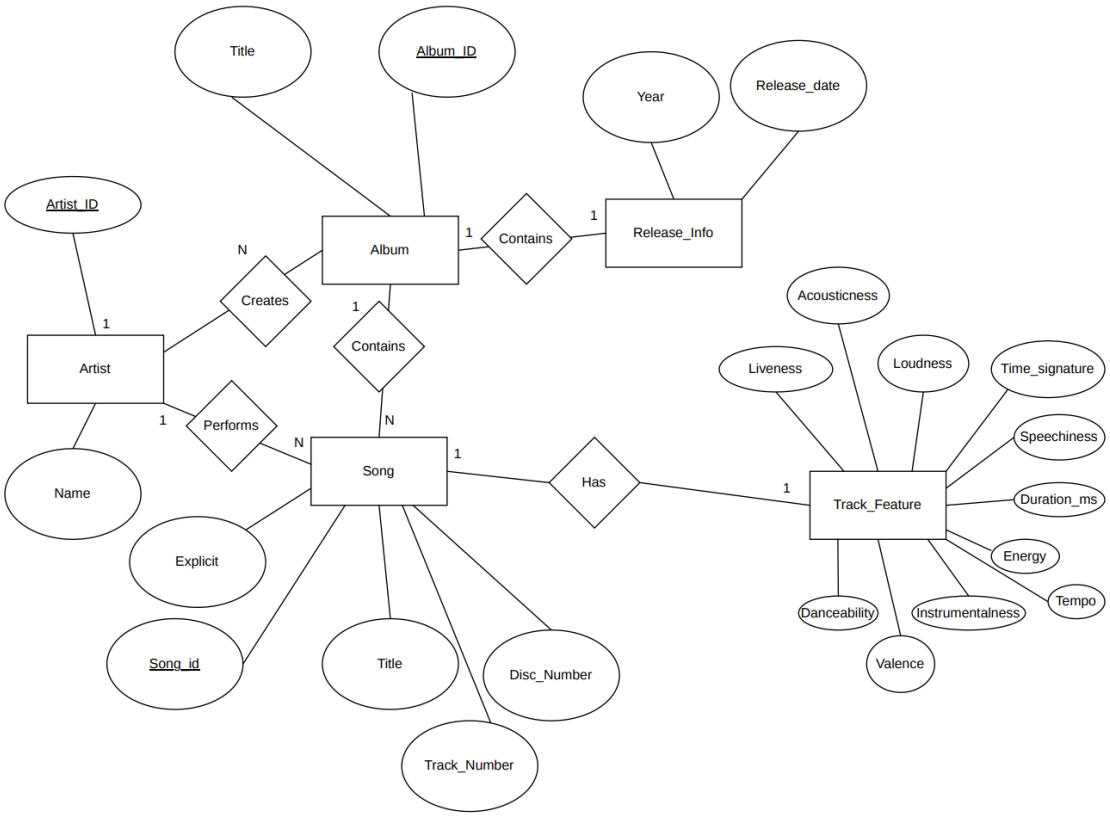
    In order to improve normalization and reduce repetition, the audio feature attributes, like danceability, energy, tempo, and duration are separated into its own Track_Feature entity, which has a one-to-one relationship with Song. The Release_Info entity also separates the release-specific information, such as release date, which has a one-to-one relationship with the Album entity.

    This design ensures that the database is normalized and scalable, while also being able to support efficient querying of both song-level and album-level characteristics. In our initial design, we included extra entities, like Genre, Listener, Playlist, and Playlist_Song. However, we decided to remove these in the final database because the dataset's information for these entities was not sufficient. Instead, we made the schema to focus more on the significant and abundant data, where we came up with Track_Feature

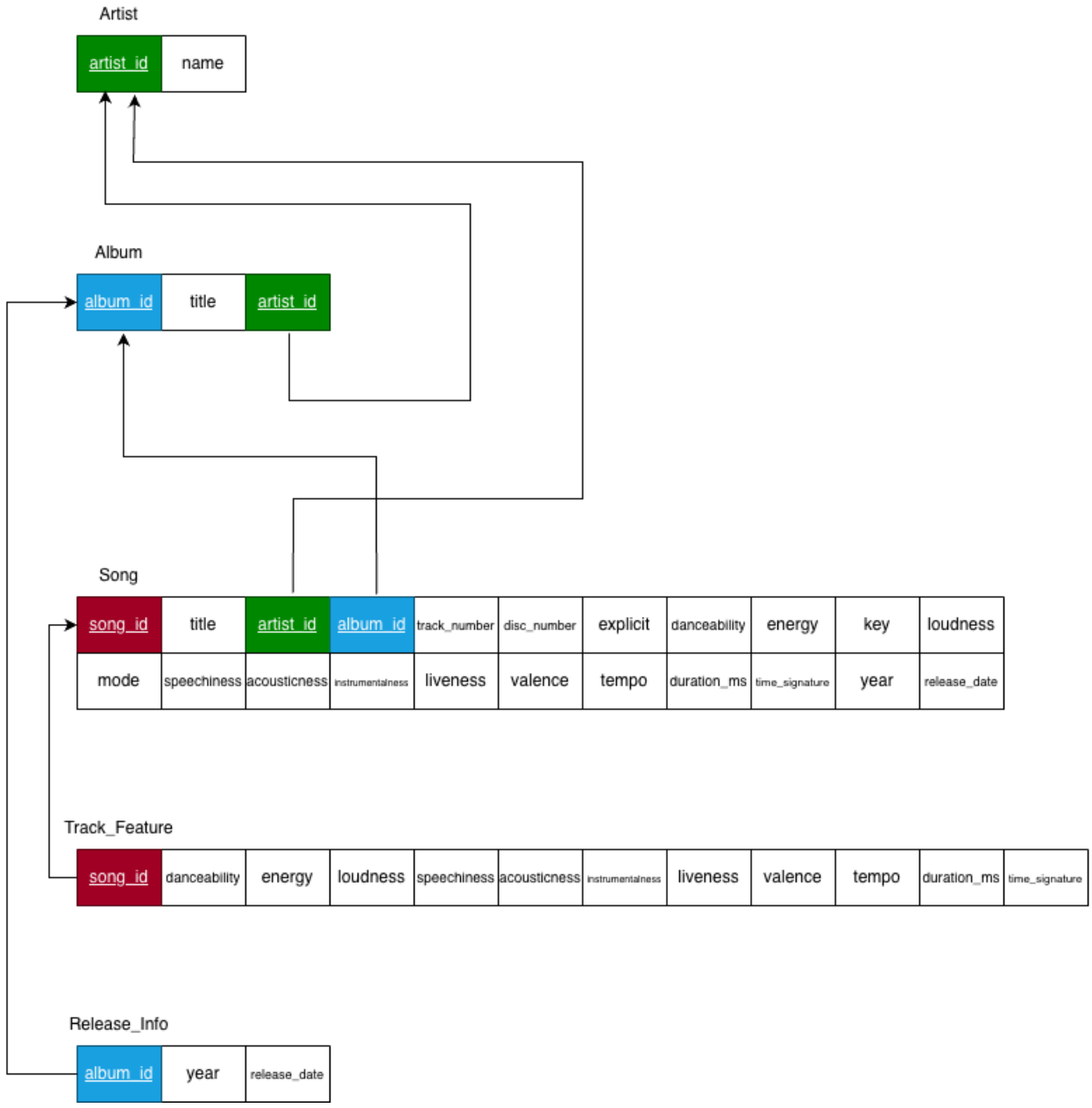
and Release_Info. These Entities normalized the dataset and separated the distinct types of information more efficiently and concisely.

- URL to database: [Spotify 1.2M+ Songs]
- https://www.kaggle.com/datasets/rodolfofigueroa/spotify-12m-songs/data

- ER Diagram:



- Relational Schema:



While the Song table holds audio feature attributes from the original dataset, a separate Track_feature table was created in order to keep those metrics more organized for analysis. Track_Feature is linked to Song through song_id, which serves as both a foreign key and its primary key, ensuring a one-to-one relationship between a song and its features. The audio metrics stored in Track_Feature cover a wide range of values. By isolating these attributes into their own table, it is easier to run feature-based queries without having to sift through the broader song metadata. While this introduces some redundancy, it makes querying a cleaner and easier process, which simplifies building analytical views around audio characteristics.

- Explanation of Roles, Keys, Foreign Keys and Attributes

Artist

artist_id	Unique key identifying the artist (unique, not null, primary key)
name	Name of Artist (not null)

Album

album_id	Unique key identifying the album (Unique, not null, Primary Key)
title	Title of the album (not null)
artist_id	Foreign key to Artist.artist_id who made the album (not null)

Song

song_id	Unique key identifying the song (Unique, not null, Primary Key)
title	Title of the song (not null)
artist_id	Foreign key to Artist.artist_id who made the song (not null)
album_id	Foreign key to Album.album_id if song is on an album
track_number	Track number of song
disc_number	Disc number of song
explicit	0 if song is not explicit, 1 if it is
danceability	Metric to gauge how related a song is to dancing
energy	Metric to gauge how energetic a song is
key	The key the song is in

loudness	How loud the song is
mode	The mode of the song
speechiness	Presence of spoken words in the song. Higher value means more spoken words, lower value means less.
acousticness	How likely it is that the song is acoustic rather than electronic
instrumentalness	Gauges how much vocals the song has. Higher value means the song has less vocals, lower value means the song has more
liveness	Gauges the presence of performers in a song. Higher value means the song feel more “live”
valence	Gauges emotion in a song. Higher value means the song is happier, lower value means the song is sadder
tempo	Tempo of the song
duration_ms	Duration of the song in ms
time_signature	Time signature of song
year	The year the song was released
release_date	The date the song was released

Track_Feature

song_id	Foreign key to Song.song_id and primary key to Track Feature (Unique, not null, Primary Key)
danceability	Metric to gauge how related a song is to dancing
energy	Metric to gauge how energetic a song is
loudness	How loud the song is
speechiness	Presence of spoken words in the song. Higher value means more spoken words, lower value

	means less.
acousticness	How likely it is that the song is acoustic rather than electronic
instrumentalness	Gauges how much vocals the song has. Higher value means the song has less vocals, lower value means the song has more
liveness	Gauges the presence of performers in a song. Higher value means the song feel more “live”
valence	Gauges emotion in a song. Higher value means the song is happier, lower value means the song is sadder
tempo	Tempo of the song
duration_ms	Duration of the song in ms
time_signature	Time signature of song

Release_Info

album_id	Foreign key to Album.album_id and primary key to Release Info (Unique, not null, Primary Key)
year	Year album was released
release_date	Date album was released

- Sizes and indexes (yellow highlight indicates we created it to optimize my queries)

Table Name	**Size (# of tuples)**	**Indexes**
Artist	166,424	PK AUTOINDEX(artist_id)
Album	118383	PK AUTOINDEX(album_id)
Song	1204025	PK AUTOINDEX(song_id) INDEX(artist_id) INDEX(album_id)
Track_Feature	1204025	INDEX(song_id)

Release_Info	118382	INDEX(album_id) INDEX(year)

- Views:

1. Listener_Song_Browse
    a. This view was made for everyday music listeners who want to browse through songs easily. It combines the song title, artist name, album title, duration, and release year into one table. This way, it eliminates the need to join multiple tables and makes the database more user-friendly when browsing and searching. This view is interesting because it acts as a song catalog that a user could search through to find songs by an artist, see what album has what songs, compare release years, or compare the length of tracks.

SQL Code:

```
CREATE VIEW Listener_Song_Browse AS
SELECT
    s.title AS song_title,
    ar.name AS artist_name,
    al.title AS album_title,
    tf.duration_ms,
    ri.year
FROM Song s
JOIN Artist ar ON s.artist_id = ar.artist_id
JOIN Album al ON s.album_id = al.album_id
JOIN Track_Feature tf ON s.song_id = tf.song_id
JOIN Release_Info ri ON al.album_id = ri.album_id;
```

View:

	song_title	artist_name	album_title
1	Testify	Rage Against The Machine	The Battle Of Los Angeles
2	Guerrilla Radio	Rage Against The Machine	The Battle Of Los Angeles
3	Calm Like a Bomb	Rage Against The Machine	The Battle Of Los Angeles
4	Mic Check	Rage Against The Machine	The Battle Of Los Angeles
5	Sleep Now In the Fire	Rage Against The Machine	The Battle Of Los Angeles

Number of Tuples:

1204025

2. Album_Summary

a. This view is designed for album-level analysis. It combines song data into album summaries, including total songs, total duration, and average energy, danceability, and valence, helping users compare albums rather than individual songs. This view is interesting because it turns millions of song rows into higher level album insights. It helps users compare albums by its length, average mood, energy level, and overall style of music. It is especially useful and efficient in studying the patterns across albums, rather than analyzing a single song at a time.

SQL Code:

```
CREATE VIEW Album_Summary AS
SELECT
    al.title AS album_title,
    ar.name AS artist_name,
    ri.year,
    COUNT(s.song_id) AS total_songs,
    SUM(tf.duration_ms) AS total_album_duration_ms,
    AVG(tf.energy) AS avg_energy,
    AVG(tf.danceability) AS avg_danceability,
    AVG(tf.valence) AS avg_valence
FROM Album al
JOIN Artist ar ON al.artist_id = ar.artist_id
JOIN Song s ON al.album_id = s.album_id
JOIN Track_Feature tf ON s.song_id = tf.song_id
JOIN Release_Info ri ON al.album_id = ri.album_id
GROUP BY al.album_id, al.title, ar.name, ri.year;
```

View:

	album_title	artist_name
1	Smart Flesh (Extras)	The Low Anthem
2	The Place To Be	Junior Cook
3	Skynut	The Gentle Men
4	No Regrets (Except for Every Decision I Have Ever Made)	Killing the Day
5	ISOHPET	OBLIQUE RAIN

year	total_song	total_albu	avg_energy	avg_danceability	avg_valence
2011	3	556023	0.388	0.	0.
1989	8	3657494	0.2828125	0.49025	0.39975
2020	1	258849	0.577	0.398	0.14
2020	1	271796	0.877	0.631	0.327
2007	7	2827827	0.917	0.	0.

Number of Tuples:

118382

3. Music_Analysis
    a. This view was made for music analysts and researchers. It focuses on audio features of songs, like the danceability, energy, instrumental, and more, allowing users to analyze the sound and characteristics of a song. This view is interesting because it analyzes the sound of songs, instead of simply the song's title or album. A user is able to compare which artists have higher-energy songs, identify songs with high danceability, or study how musical features vary across different artists and albums.

SQL Code:

```
CREATE VIEW Music_Analysis AS
SELECT
    s.title AS song_title,
    ar.name AS artist_name,
    al.title AS album_title,
    tf.danceability,
    tf.energy,
    tf.valence,
    tf.tempo,
    tf.acousticness,
    tf.instrumentalness
FROM Song s
JOIN Artist ar ON s.artist_id = ar.artist_id
JOIN Album al ON s.album_id = al.album_id
JOIN Track_Feature tf ON s.song_id = tf.song_id;
```

View:

	song_title	artist_name	album_title
1	Testify	Rage Against The Machine	The Battle Of Los Angeles
2	Guerrilla Radio	Rage Against The Machine	The Battle Of Los Angeles
3	Calm Like a Bomb	Rage Against The Machine	The Battle Of Los Angeles
4	Mic Check	Rage Against The Machine	The Battle Of Los Angeles
5	Sleep Now In the Fire	Rage Against The Machine	The Battle Of Los Angeles

danceabili	energy	valence	tempo	acousticnes	instrumenta
0.47	0.978	0.503	117.906	0.0261	0.0000109
0.599	0.957	0.489	103.68	0.0129	0.0000706
0.315	0.97	0.37	149.749	0.0234	0.00000203
0.44	0.967	0.574	96.752	0.163	0.00000364
0.426	0.929	0.539	127.059	0.00162	0.105

Number of Tuples:

1204025

- Documented Queries:

Query 1: List all the songs in the dataset of the top 100 artists, along with their length and albums

This query identifies the top 100 artists in the dataset and then shows a sample of their individual songs. The output shows the song title, the artist's name, the title of the album the song belongs to, and the duration in milliseconds of the song. In the sample output, one of the top artists is Blue Öyster Cult with songs from the album Mirrors.

This query uses a subquery within the where clause, which groups the records by artist_id, counts how often they show up, and orders them descending to find the top 100 artists with the most songs. The main query joins the tables Song, Artist, Album, and Track_Feature, where it filters the results to include the artists from the subquery.

SQL Code:

```
SELECT
    s.title AS song_title,
    ar.name AS artist_name,
    al.title AS album_title,
    tf.duration_ms
FROM Song s
JOIN Artist ar ON s.artist_id = ar.artist_id
JOIN Album al ON s.album_id = al.album_id
JOIN Track_Feature tf ON s.song_id = tf.song_id
```

```
WHERE ar.artist_id IN (
    SELECT s2.artist_id
    FROM Song s2
    GROUP BY s2.artist_id
    ORDER BY COUNT(*) DESC
    LIMIT 100
)
LIMIT 50;
```

Sample Output:

	song_title	artist_name	album_title	duration_ms
1	Dr. Music	Blue Ã–yster Cult	Mirrors	193707
2	The Great Sun Jester	Blue Ã–yster Cult	Mirrors	290853
3	In Thee	Blue Ã–yster Cult	Mirrors	230147
4	Mirrors	Blue Ã–yster Cult	Mirrors	227800
5	Moon Crazy	Blue Ã–yster Cult	Mirrors	247440

Query 2: Find the total number of songs that are associated with each artist, excluding the "various artists" found in the dataset

This query counts the number of songs each artist in the database is associated with, while filtering out a generic label, specifically “Various Artists”, to ensure we get only useful information. The query displays both the artists’ names and the number of songs they are a part of in descending order. In the output, the artist who appears in the most songs is “Vitamin String Quartet”, with 1577.

The query uses a standard JOIN with the Artist and Song tables based on the artist_id and a group by clause on artist_id and name. The COUNT function lists the total number of tracks per artist, which is then sorted using ORDER BY.

SQL Code:

```
SELECT
    ar.name AS artist_name,
    COUNT(s.song_id) AS total_songs
FROM Artist ar
JOIN Song s ON ar.artist_id = s.artist_id
WHERE ar.name != 'Various Artists'
GROUP BY ar.artist_id, ar.name
ORDER BY total_songs DESC
LIMIT 50;
```

Sample Output:

	artist_name	total_songs
1	Vitamin String Quartet	1577
2	Aretha Franklin	1209
3	The City of Prague Philharmonic Orchestra	1042
4	"Dan Gibsons Solitudes"	997
5	Bob Dylan	913
6	The Fall	812
7	"Pickin On Series"	774
8	Dolly Parton	747
9	Guided By Voices	739
10	Dionne Warwick	728

Query 3: Find the total number of songs that each album contains

This query counts the total number of songs on every album within the dataset, which allows us to clearly see the sizes of albums. The output of the query is the album title, the artist's name, and the number of songs on said album.

The tables used are the Album, Artist, and Song tables, which are joined using artist_id and album_id. The WHERE clause removes "Various Artists" to eliminate a generic term. The query uses GROUP BY on the album_id, album name, and artist name. From there, the query COUNTS each s.song_id, effectively counting the number of songs in every unique album. The results are then ordered in descending order. In the output, we can see that the top 5 albums with the most songs are all tied with 50 songs.

SQL Code:

```
SELECT
    al.title AS album_title,
    ar.name AS artist_name,
    COUNT(s.song_id) AS total_songs
FROM Album al
JOIN Artist ar ON al.artist_id = ar.artist_id
JOIN Song s ON al.album_id = s.album_id
WHERE ar.name != 'Various Artists'
```

```
GROUP BY al.album_id, al.title, ar.name
ORDER BY total_songs DESC
LIMIT 50;
```

Sample Output:

	album_title	artist_name	total_songs
1	Mozart: Ascanio in Alba	Wolfgang Amadeus Mozart, RaphaÃ«l Boulay, Michael Chance, Jill Feldman, Rosa Mannion, Howard ...	50
2	Johann Sebastian Bach: MatthÃ¤us-Passion	Gewandhausorchester Leipzig, Dresdner Kreuzchor, Thomanerchor Leipzig, Erhard Mauersberger, ...	50
3	Beethoven: The Complete Piano Sonatas	Ludwig van Beethoven, Igor Levit	50
4	The Man With The Mandolin	Dave Apollon	50
5	Dead Man's Pop	The Replacements	50

Query 4 : Find the total length of the album based off the length of each song in the albums

This query calculates the total duration in milliseconds of each album in its entirety. The output is the album_title, artist_name, total_songs, and total_album_duration_ms, which is the individual durations of all of its tracks, while displaying the longest albums from greatest to least.

The query merges the Album, Artist, Song, and Track_Feature using JOINs. The query groups the data by album_id, title, and artist_name to get every unique album. Then it uses two aggregate functions, COUNT and SUM. COUNT(s.song_id) counts the number of songs in the album, and SUM(tf.duration_ms) adds the duration of each song in the album. ORDER BY sorts the final output in descending order by total album duration.

SQL Code:

```
SELECT
    al.title AS album_title,
    ar.name AS artist_name,
    COUNT(s.song_id) AS total_songs,
    SUM(tf.duration_ms) AS total_album_duration_ms
FROM Album al
JOIN Artist ar ON al.artist_id = ar.artist_id
JOIN Song s ON al.album_id = s.album_id
JOIN Track_Feature tf ON s.song_id = tf.song_id
GROUP BY al.album_id, al.title, ar.name
ORDER BY total_album_duration_ms DESC
LIMIT 50;
```

Sample Output:

	album_title	artist_name	total_songs	total_album_duration_ms
1	My Life's Work	Ben Seretan	48	86400000
2	Ecobag: 13 Japanese Birds In A Bag	Merzbow	43	43325523
3	Mahler: Symphonies Nos. 1-9	Gustav Mahler, Philharmonia Orchestra, Lorin Maazel	50	39158282
4	Michael Gielen Edition, Vol. 6: Mahler Symphonies & Orchestral Song Cycles (Recorded 1988-2014)	Gustav Mahler, SWR Symphony Orchestra, Michael Gielen	50	36355077
5	Bruckner: Symphonies Nos. 1-9	Anton Bruckner, Orchestre MÃ©tropolitain, Yannick NÃ©zet-SÃ©guin	35	36337481
6	9 Compositions (Iridium) 2006	Anthony Braxton	35	34965149

Query 5 : Find the average danceability and valence of songs released each year listed

This query analyzes the features of the songs over time, focusing on the danceability and valence of the tracks. The output displays the release year, the number of songs released that year, and the calculated average scores for danceability and valence. This query provides insight into historical shifts in the mood and rhythm of music by year.

The query connects the Release_Info, Album, Song, and Track_Feature tables, connecting the release dates of songs with the metadata of each song. Then groups and orders the dataset chronologically by ri.year. The COUNT function tracks the number of songs released that year, while the AVG function is used to calculate the mean danceability and valence. The results are then displayed chronologically by year in ascending order.

SQL Code:

```
SELECT
    ri.year,
    COUNT(s.song_id) AS total_songs,
    AVG(tf.danceability) AS avg_danceability,
    AVG(tf.valence) AS avg_valence
FROM Release_Info ri
JOIN Album al ON ri.album_id = al.album_id
JOIN Song s ON al.album_id = s.album_id
JOIN Track_Feature tf ON s.song_id = tf.song_id
GROUP BY ri.year
ORDER BY ri.year
LIMIT 50;
```

Sample Output:

	year	total_songs	avg_danceability	avg_valence
1	0	10	0.7655	0.6546
2	1900	14	0.483857142857143	0.422157142857143
3	1908	19	0.604105263157895	0.657
4	1909	25	0.61756	0.82388
5	1917	52	0.506942307692308	0.480892307692308
6	1920	25	0.5626	0.62024
7	1923	50	0.62888	0.67606
8	1924	36	0.639138888888889	0.764027777777778
9	1925	49	0.503163265306122	0.442214285714286
10	1926	107	0.636	0.652747663551402

Query 6: Find the albums with the highest energy rating across the songs in them

This query identifies the albums with the highest average energy. The query averages the energy metric of all individual tracks in the album and outputs the album title, artist's name, total number of songs, and the average energy rating of the album.

The query joins the Album, Artist, Song, and Track_Feature tables using artist_id, album_id, and song_id. The records are then grouped into their unique album by using album_id, the album title, and the artist name. The HAVING clause filters the results and excludes any album that does not have at least 5 songs, ensuring that any singles or EPs do not skew the averages. The AVG(tf.energy) function gives the mean energy score, and the results are then sorted in descending order using this average.

SQL Code:

```
SELECT
    al.title AS album_title,
    ar.name AS artist_name,
    COUNT(s.song_id) AS total_songs,
    AVG(tf.energy) AS avg_energy
FROM Album al
JOIN Artist ar ON al.artist_id = ar.artist_id
```

```
JOIN Song s ON al.album_id = s.album_id
JOIN Track_Feature tf ON s.song_id = tf.song_id
GROUP BY al.album_id, al.title, ar.name
HAVING COUNT(s.song_id) >= 5
ORDER BY avg_energy DESC
LIMIT 50;
```

Sample Output:

	album_title	artist_name	total_songs	avg_energy
1	Wind Chime Waterfall	Global Journey	11	1
2	Black Moldy Brew	Lair	10	0.999
3	Dingir	Rings of Saturn	10	0.9986
4	Hellbringer	Exodia	10	0.9984
5	Rummel	Y8K, Itt0	5	0.998
6	Alive Again	Nuclear Assault	12	0.997833333333333

Query 7: Find the artists who have the highest average danceability

In query 7, it finds the artists with the highest average danceability across their songs. It joins the Artist, Song, and Track_Features tables, then groups by artists to calculate the artist's total songs and their average danceability. The HAVING COUNT (s.song_id) >= 10 condition makes sure that only artists with 10 songs or more are included, making the results more meaningful. The final output is sorted from highest to lowest average danceability.

SQL Code:

```
SELECT
    ar.name AS artist_name,
    COUNT(s.song_id) AS total_songs,
    AVG(tf.danceability) AS avg_danceability
FROM Artist ar
JOIN Song s ON ar.artist_id = s.artist_id
JOIN Track_Feature tf ON s.song_id = tf.song_id
GROUP BY ar.artist_id, ar.name
HAVING COUNT(s.song_id) >= 10
ORDER BY avg_danceability DESC
LIMIT 50;
```

Sample Output:

	artist_name	total_songs	avg_danceability
1	Yehjah & Go a Chant	10	0.9373
2	Monchy & Alexandra, Alexandra	12	0.929166666666667
3	M.C. Shy-D	17	0.922529411764706
4	El Gringo De La Bachata	10	0.9097
5	KenTheMan	10	0.9052
6	De-U Records	20	0.901

Query 8 : List songs above a certain energy threshold that came out each year

In query 8, it shows how many high-energy songs were released in each year. It joins Release_Info, Album, Song, and Track_Feature, then filters for songs with energy greater than or equal to 0.85. It groups the results by year and shows the number of high-energy songs and the average energy for that year. This helps identify which years had the most energy in their music on the dataset.

SQL Code:

```
SELECT
    ri.year,
    COUNT(s.song_id) AS high_energy_song_count,
    AVG(tf.energy) AS avg_energy
FROM Release_Info ri
JOIN Album al ON ri.album_id = al.album_id
JOIN Song s ON al.album_id = s.album_id
JOIN Track_Feature tf ON s.song_id = tf.song_id
WHERE tf.energy >= 0.85
GROUP BY ri.year
ORDER BY high_energy_song_count DESC
LIMIT 50;
```

Sample Output:

	year	high_energy_song_count	avg_energy
1	2019	12817	0.924017788874152
2	2020	11928	0.923252682763246
3	2018	10890	0.924128191000918
4	2017	9690	0.923975128998968
5	2007	9544	0.92359513830679
6	2006	9331	0.921865287750509

Query 9 : Find the average tempo of songs for each artist

This query calculates the average tempo of the songs for each artist. It joins the Artist, Song, and Track_Feature tables. Then, it groups the results by artist. It also counts how many songs an artist has and only includes artists with a t least 10 songs to make sure the averages are meaningful. The results are sorted from highest to lowest average tempo, which allows the users to identify which artists tend to produce faster music tempos.

SQL Code:

```
SELECT
    ar.name AS artist_name,
    COUNT(s.song_id) AS total_songs,
    AVG(tf.tempo) AS avg_tempo
FROM Artist ar
JOIN Song s ON ar.artist_id = s.artist_id
JOIN Track_Feature tf ON s.song_id = tf.song_id
GROUP BY ar.artist_id, ar.name
HAVING COUNT(s.song_id) >= 10
ORDER BY avg_tempo DESC
LIMIT 50;
```

Sample Output:

	artist_name	total_songs	avg_tempo
1	Zombified	10	178.5822
2	Steven Halpern, The Sound Medicine Band, Chief Sonne Reyna	12	174.49025
3	Think Again	10	174.2559
4	Bustre	10	174.0082
5	Logistics	18	173.733944444444

Query 10 : Find the songs that have a high energy but low acousticness in their songs

This query finds songs that have high energy, but low acousticness, which means that songs are energetic, but not acoustic. It joins the Song, Artist, Album, and Track_Feature tables, then filters for songs with energy greater than or equal to 0.85 and acousticness less than or equal to 0.1. The results are sorted by energy in descending order, highlighting the most intense songs that meet these conditions.

SQL Code:

```
SELECT
    s.title AS song_title,
    ar.name AS artist_name,
    al.title AS album_title,
    tf.energy,
    tf.acousticness
FROM Song s
JOIN Artist ar ON s.artist_id = ar.artist_id
JOIN Album al ON s.album_id = al.album_id
JOIN Track_Feature tf ON s.song_id = tf.song_id
WHERE tf.energy >= 0.85
  AND tf.acousticness <= 0.10
ORDER BY tf.energy DESC
LIMIT 50;
```

Sample Output:

	song_title	artist_name	album_title	energy	acousticness
1	Einstein-Rosen Bridge	Venetian Snares	The Chocolate Wheelchair Album	1	0.0813
2	Soft Water Rhinoceros	Merzbow	Tauromachine	1	0.0285
3	Bumba (A Buddha)	Robert GÃ¶rl	Watch the Great Copy Cat	1	0.000156
4	Skald Au Satans Sol	Darkthrone	Transilvanian Hunger (20th Anniversary Edition)	1	0.098
5	Say No To Emo	ESA	Devotion, Discipline, And Denial	1	0.000431

Query 11 : Find the artists whose songs span the greatest number of albums

This query finds the artists whose songs span the greatest number of albums. It joins the Artist, Song, and Album tables, then counts the number of distinct albums each artist appears on, along with their total number of songs. The results are sorted by artist and in descending order by the number of albums, This helps identify artists who have the widest range of songs across different albums, which may indicate high productivity or collaborations.

SQL Code:

```
SELECT
```

```
    ar.name AS artist_name,
    COUNT(DISTINCT al.album_id) AS total_albums,
    COUNT(s.song_id) AS total_songs
FROM Artist ar
JOIN Song s ON ar.artist_id = s.artist_id
JOIN Album al ON s.album_id = al.album_id
GROUP BY ar.artist_id, ar.name
ORDER BY total_albums DESC
LIMIT 50;
```

Sample Output:

	artist_name	total_albums	total_songs
1	Grant Macdonald	403	587
2	Monstercat Call of the Wild	310	310
3	Various Artists	170	1753
4	Vitamin String Quartet	152	1577
5	Foreigner	137	216

Query 12 : Find the songs who have the widest range of energy values

This query identifies the artists whose songs have the widest range of energy values. It joins together the Artist, Song, and Track_Feature tables, then calculates the minimum and maximum energy for each artist's songs and gets the difference between them, in terms of the energy range. Only artists with at least 10 songs are included to ensure significant comparisons. The results are sorted in descending order by energy range, highlighting artists with the most variation in the musical intensity across their songs.

SQL Code:

```
SELECT
    ar.name AS artist_name,
    COUNT(s.song_id) AS total_songs,
    MIN(tf.energy) AS min_energy,
    MAX(tf.energy) AS max_energy,
    MAX(tf.energy) - MIN(tf.energy) AS energy_range
FROM Artist ar
JOIN Song s ON ar.artist_id = s.artist_id
JOIN Track_Feature tf ON s.song_id = tf.song_id
GROUP BY ar.artist_id, ar.name
HAVING COUNT(s.song_id) >= 10
```

ORDER BY energy_range DESC
LIMIT 50;

Sample Output:

	artist_name	total_songs	min_energy	max_energy	energy_range
1	Sound Collective	100	0	1	1
2	Benny Goodman	268	0	1	1
3	The BBC Radiophonic Workshop, Brian Hodgson	23	0	1	1
4	C/A/T	58	0	1	1
5	N/A	10	0	1	1
6	Cattle Decapitation	49	0	1	1
7	Sissy Spacek	71	0.0000186	1	0.9999814
8	Dwarves	88	0.0000202	1	0.9999798